Compte Rendu final de Projet de Recherche

Pierre Baumann, Ivan Mulot-Radojcic, Damien Assire

12 mai 2024

Résumé

Abstract

Table des matières

1 Introduction **1**

2 Génération du graphe **2**
2.1 Transformation du graphe . . . 3

3 Génération des attaques **5**
3.1 Attaques statiques . . . 5
3.1.1 Attaque aléatoire statique . . . 5
3.1.2 Attaque par centralité intermédiaire . . . 5
3.2 Attaques dynamiques . . . 6
3.2.1 Attaque complètement aléatoire . . . 6
3.2.2 Attaque aléatoire mouvante . . . 6
3.2.3 Attaque par centralité intermédiaire mouvante . . . 7

4 Calcul de la mesure **7**
4.1 Premier algorithme . . . 7
4.2 Algorithme amélioré . . . 8
4.2.1 Création d'une structure de données adaptée . . . 8
4.2.2 Algorithme de calcul de la mesure . . . 9
4.2.3 Optimisations faites . . . 9
4.2.4 Complexité de l'algorithme . . . 11
4.3 Autres optimisations tentées : Invalidation des chemins attaqués . . . 11

5 Résultats et interprétation **13**

6 Perspectives et ouvertures **13**

7 Conclusion **14**

sections

1 Introduction

Dans notre projet, nous cherchons à modéliser et évaluer l'impact d'une manifestation sur une ville. Pour modéliser ce problème nous utiliserons une variante des graphes, les graphes dynamiques.

Un graphe est un objet informatique constitué de sommets reliés par des arrêtes. Les graphes permettent de modéliser de nombreux problèmes, notamment en informatique, en biologigie, en sociologie, physique,...

Un graphe peut être défini par $G = (V, E)$ où V est l'ensemble des sommets et $E \subseteq V^2$ est l'ensemble des liens $e = (u, v, w)$ $u, v \in V, w \in \mathbb{N}$ qui est la pondération du lien de u à v, noté u $\xrightarrow{\text{w}}$ v. Les graphes seront également orienté, i.e si $(u, v, w) \in E$ alors il est possible d'accéder à v depuis u mais pas forcément d'accéder à u depuis v, à moins que $\exists\ (v, u, w') \in E$.

Un graphe dynamique est alors un graphe dont les sommets et les arrêtes peuvent évoluer dans le temps. Le modèle de graphe dynamique que nous utilisons peut être décrit comme un triplet $G = (V, E, T)$ où $T = \{(t, E_t) \mid t \in [\![0, t_{max}]\!] \text{ et } E_t \subseteq E\}$ est l'ensemble des instants auxquelles des liens entre les sommets sont supprimés.

référence pour graphe dynamique

Nous utilisons également diamètre d'un graphe, définie par :

$$diametre = \max_{\forall u,v \in V} \{\text{longueur}(PlusCourtChemin(u, v))\} \text{ [1]}$$

La mesure classique pour déterminer à quel point il est facile de se déplacer dans un réseau est l'efficacité. L'efficacité d'un graphe est définie comme la moyenne des inverses des distances entre chaque paire de sommets.

$$E(G) = \frac{1}{N(N-1)} \sum_{i \neq j} \frac{1}{d(v_i, v_j)} \tag{1}$$

avec $d(v_i, v_j)$ la distance entre deux sommets v_i et v_j, N le nombre de sommets du graphe.

référence pour efficacité

Notre travail est alors de déterminer une mesure de l'efficacité dans un graphe dynamique et de trouver des stratégies pour attaquer ce graphe et réduire cette efficacité en prenant en compte que le graphe est dynamique et va évoluer dans le temps.

2 Génération du graphe

Nous représentons les villes sous la forme d'un graphe. Les données sont importées depuis une bibliothèque de données de rues nommées OpenStreetMap [2]. Pour cela, nous utilisons la librairie Python OSMnx [3] qui les convertis sous la forme d'un graphe.

1. Cette formule ne marche que si le graphe est connexe, sinon il faut prendre le maximum des diamètres des composantes connexes.
2. https://www.openstreetmap.org
3. https://osmnx.readthedocs.io/en/stable/

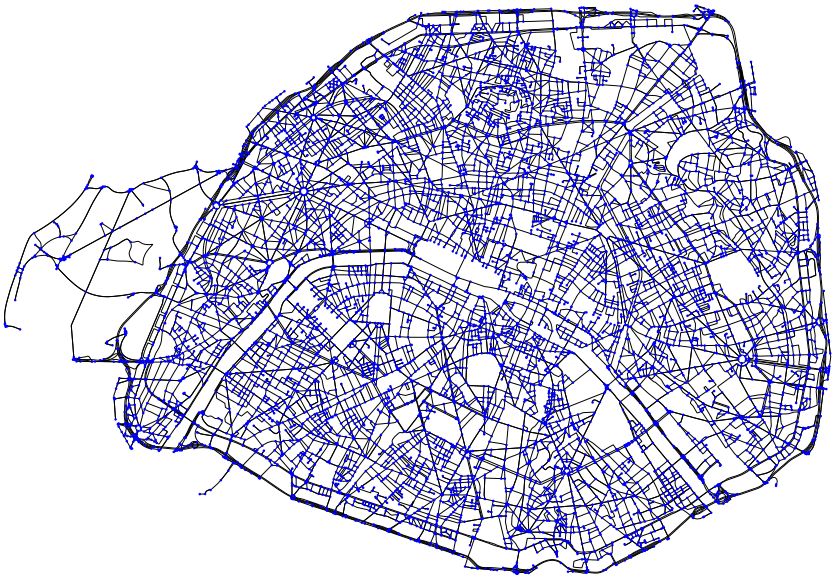


FIGURE 1 – Graphe de la ville de Paris

2.1 Transformation du graphe

Nous avons choisi de représenter le graphe sous la forme d'une matrice creuse en format CSR (Compressed Sparse Row). Sous cette représentation, le graphe est représenté par trois tableaux :

— ROW_INDEX qui contient les bornes des intervalles de valeurs liés aux liens partant d'un nud u.

$$[\![ROW_INDEX[u], ROW_INDEX[u+1]-1]\!]$$

Si u n'a aucun lien sortant, $ROW_INDEX[u] = ROW_INDEX[u+1]$.

— COL_INDEX contient le sommet destination d'un lien à l'index donné

— V contient le poids du lien.

Lors de la récupération des données, OSMnx renvoie un multi-graphe orienté.

Nous transformons le graphe car :

1. Nous utilisons des distances temporels et non la distance physique entre les deux noeuds.
2. La ville est représentée sous la forme d'un multigraphe alors que les simulations sont faites sur un graphe avec au plus un lien d'un sommet u à v.

Pour cela, nous utilisons les fonctions des librairies NetworkX et SciPy afin d'obtenir les tableaux ROW_INDEX et COL_INDEX. Le tableau V doit est calculé via l'algorithme 1

Algorithm 1 Calcul de V

Require : G : multi-graphe, ROW_INDEX, COL_INDEX
  for $i \in [\![0, G.NumNodes]\!]$ **do**
    for $j \in ROW_INDEX[i, i+1]$ **do**
      $weigth \leftarrow \emptyset$
      for all $e \in G.edges[i][COL_INDEX[j]]$ **do**
        $weigth.append\left(\frac{e.length}{e.maxspeed}\right)$
      end for
      $V[j] \leftarrow min(weigth)$
    end for
  end for

L'algorithme 1 parcourt l'ensemble des couples $(u, v) \in V^2$ tels qu'il existe au moins un lien de u à v dans E. Après, il parcourt tout les liens de u à v et calcul le minimum des distances temporelles, donnés par la formule

$$\Delta t = \frac{d}{v}$$

Où la durée Δt est la distance temporelle, d la longueur du lien et v la vitesse maximale. Dans le cas où il y a plusieurs liens entre deux sommets u et v, on garde le lien de poids le plus petit car c'est celui-ci qui sera emprunter lors d'un plus court chemin. Certaines données nécessaires aux calculs peuvent êtres manquantes et sont remplacées par des valeurs par défaut pour la vitesse max, et la distance euclidienne de u à v pour la longueur du lien.

La figure 2 montre le graphe résultant de cette transformation sur un sous-ensemble du graphe de Paris correspondant au cinquième arrondissement.

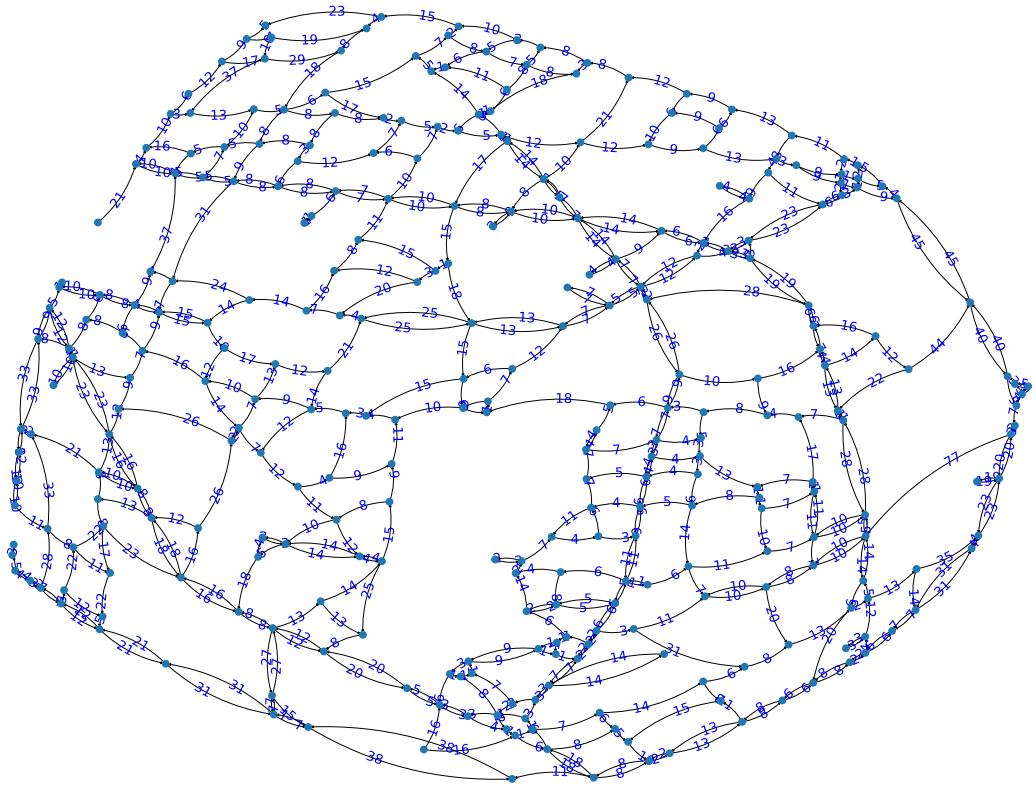


FIGURE 2 – Transformation en graphe temporel du 5e arrondissement de Paris

3 Génération des attaques

À chaque pas de temps t_i, nous allons attaquer le graphe en bloquant des liens jusqu'à un certain budget. Nous avons choisi de mettre le prix de chaque lien à 1, car il est plus simple d'interpréter les résultats avec un prix constant.

Nous avons choisi de séparer la gestion des attaques des stratégies d'attaques. Pour la gestion, nous avons la classe Attaque, avec 2 primitives :

— Bloquer un lien à un temps donné

— Retranscrire l'attaque dans un fichier pour la simulation

Comme ça, les stratégies peuvent se focaliser sur l'algorithme de choix des liens à supprimer, et la classe Attaque se charge de faire le reste.

Nous classons les stratégies d'attaques en deux catégories : les attaques dynamiques et les attaques statiques.

3.1 Attaques statiques

Les attaques statiques sont des attaques qui n'évoluent pas dans le temps, c'est à dire qu'un lien bloqué est bloqué de t_0 à $t_{\max}$.

3.1.1 Attaque aléatoire statique

Cette première stratégie consiste à bloquer des liens aléatoirement, et rester sur ces liens pour toute la durée de l'attaque.

Algorithm 2 Attaque aléatoire statique

```
Liens ← Graphe.liens()
LiensBloques ← Mélanger(Liens)
for lien ← LiensBloques do
    Attaque.bloquer(lien, t)
end for
```

3.1.2 Attaque par centralité intermédiaire

Dans l'état-de-l'art, l'attaque par centralité intermédiaire est considérée comme la stratégie la plus efficace. La centralité intermédiaire est une mesure représentant l'importance d'un lien dans un graphe. Elle est calculée en comptant le nombre de plus courts chemins passant par ce lien.

Cette stratégie consiste donc à bloquer les liens les plus importants en les triant par centralité intermédiaire. Nous notons d'ailleurs qu'OSMnx contient des options pour récupérer les centralités intermédiaires des liens d'un graphe directement.

Algorithm 3 Attaque par centralité intermédiaire

```
Liens ← Graphe.liens()
LiensTries ← TrierParCentralité(Liens)
for t ← 0 to TempsMax do
    BudgetRestant ← Budget
    for lien ← LiensTries do
        Attaque.bloquer(lien, t)
        Prix ← lien.prix()
        BudgetRestant ← BudgetRestant − Prix
        if BudgetRestant ≤ 0 then
            break
        end if
    end for
end for
```

Remarque : cette stratégie est en fait une stratégie générique, qui peut être utilisée pour trier les liens par n'importe quel critère, comme le nombre de voisins, le nombre de voies composant la rue, etc... Nous l'utilisons avec la centralité intermédiaire, car c'est le critère le plus couramment utilisé dans la littérature.

3.2 Attaques dynamiques

Les attaques dynamiques sont des attaques où un lien peut être bloqué puis débloqué plus tard, notamment, car les obstacles se déplacent au cours du temps. Nous avons choisi d'étudier des équivalents des attaques statiques, mais qui évoluent dans le temps, pour voir si elles sont plus efficaces.

3.2.1 Attaque complètement aléatoire

Cette stratégie est la première adaptation dynamique de l'attaque aléatoire. Elle consiste à attaquer des liens aléatoirement, différents à chaque pas de temps. Cette attaque nous servira de base pour comparer les autres stratégies.

Algorithm 4 Attaque complètement aléatoire

```
for t ← 0 to TempsMax do
    LiensRestants ← Mélanger(Graphe.liens())
    BudgetRestant ← Budget
    while BudgetRestant > 0 ∧ LiensRestants ≠ ∅ do
        lienABloquer ← Liens.pop()
        Attaque.bloquer(lienABloquer, t)
        Prix ← lienABloquer.prix()
        BudgetRestant ← BudgetRestant − Prix
    end while
end for
```

3.2.2 Attaque aléatoire mouvante

Pour cette stratégie, nous commençons par une attaque aléatoire pour le premier pas de temps t_0, mais pour les suivants, les obstacles bloquant un lien vont se déplacer vers un lien voisin non bloqué s'ils le peuvent. Dans notre contexte de manifestations et de marches, cette stratégie est plus réaliste, car les manifestants vont se déplacer dans la ville.

Algorithm 5 Attaque mouvante

```
LiensOriginaux ← AttaqueAléatoire(Graphe, t_max=1, Budget)
for t ← 1 to t_max do
    NewLiens ← ∅
    for lien ← LiensOriginaux do
        Voisins ← lien.voisins()
        for v ← Voisins do
            if v ∉ NewLiens then
                NewLiens.bloquer(v)
                continue to next lien
            end if
        end for
        NewLiens.bloquer(lien)
    end for
    LiensOriginaux ← NewLiens
end for
```

3.2.3 Attaque par centralité intermédiaire mouvante

Cette stratégie consiste à bloquer les liens les plus importants en les triant par centralité intermédiaire, puis pour les prochains pas de temps, on calcule les nouvelles centralités intermédiaires sur le graphe avec les liens bloqués, et on bloque ces nouveaux liens les plus importants. Note : dû aux nombre de calculs nécessaires pour les centralités intermédiaires, nous les avons approximées en ne prennant que 20% des chemins les plus courts, ce qui est suffisant pour avoir une très bonne approximation.

Algorithm 6 Attaque par centralité intermédiaire mouvante

```
Liens ← Graphe.liens()
LiensTries ← TrierParCentralité(Liens)
LiensBloques ← ∅
for t ← 0 to TempsMax do
    BudgetRestant ← Budget
    for lien ← LiensTries do
        Attaque.bloquer(lien, t)
        LiensBloques.ajouter(lien)
        Prix ← lien.prix()
        BudgetRestant ← BudgetRestant − Prix
        if BudgetRestant ≤ 0 then
            break
        end if
    end for
    Graphe = Graphe.enlever(LiensBloques)
    LiensTries ← TrierParCentralité(Graphe.liens())
end for
```

4 Calcul de la mesure

4.1 Premier algorithme

Le problème des plus courts chemins est déjà assez complexe dans un graphe classique, mais il devient encore plus difficile dans un graphe temporel, où les arêtes peuvent être bloquées à certains moments. Notre première approche a été d'adapter l'algorithme de Floyd-Warshall. Ce choix a été motivé par le fait que l'algorithme est simple à comprendre et à adapter a un graphe temporel. Nous avons donc implémenté cette algorithme en C pour effectuer nos premières mesures. Nous nous sommes cependant rendu compte que le programme de simulation n'était pas assez efficace, notamment à cause de la complexité de l'algorithme de Floyd-Warshall, qui est en $O(n^3)$, et qui devient en $O(n^3 \times t)$ pour un graphe temporel, avec n le nombre de noeuds et t le nombre de "pas de temps" dans le graphe temporel. Nous avons donc décidé de changer d'approche et de chercher algorithme plus efficace.

Le problème des calculs de chemin dans les graphes temporels

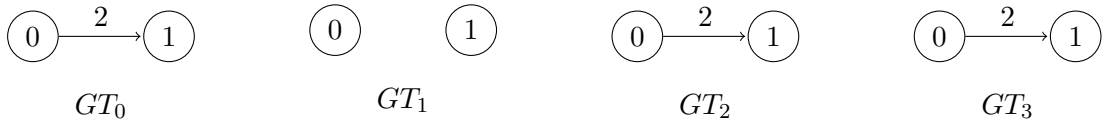
```
TemporalFloydWarshall(graphe temporel GT, DistanceMatrixes dma)
  InitTFW(GT, dma)
  for time in {dma.NumMatrixes .. 0} do
    for k in {0 .. |GT.V| } do
      for u in {0 .. |GT.V| } do
        for v in {0 .. |GT.V| } do
          distUV ← dma.matrix[t_i][u][v]
          distUK ← dma.matrix[t_i][u][k]
          distKV ← dma.matrix[t_i + distUK][k][v]
          if distUV > distUK + distKV then
            dma.matrix[t_i][u][v] ← distUK + distKV
          end if
        end for
      end for
    end for
  end for
```

4.2 Algorithme amélioré

4.2.1 Création d'une structure de données adaptée

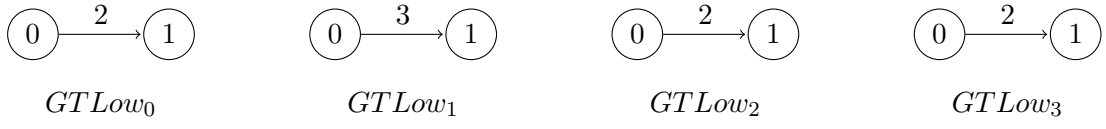
Le problème des autres algorithmes classiques de graphes est qu'ils assument que le graphe est statique, et donc ne peuvent pas prendre en compte les liens bloqués puis débloqués. Cependant, nous avons connaissance du graphe à tout instant, et nous avons donc pensé à créer des liens qui remplacent les liens supprimés, équivalents à la distance du lien d'origine plus l'attente nécessaire pour que le lien soit à nouveau disponible, que nous appelons "liens fantômes". Cela va permettre aux algorithmes de ne pas avoir à prendre en compte l'évolution du graphe pour calculer les chemins, tout en calculant quand même la vraie distance minimale, car le cas où attendre est bénéfique a été pris en compte par le lien fantôme. Ainsi, nous avons pu rendre beaucoup plus de parcours de graphes adaptables temporellement, notamment Dijkstra, que nous avons utilisé pour notre implémentation.

Cependant, calculer ces liens fantômes relève de l'interprétation de l'impact qu'à le blocage d'un lien. Ici nous définissons deux impacts possibles pour un lien de poids p bloqué à un temps t : Un impact bas signifie que pour traverser le lien, il faut juste l'atteindre, c'est-à-dire que si le lien existe au temps t, alors on peut le traverser. Un impact haut signifie que pour traverser le lien, il faut qu'il soit disponible à chaque instant où on le traverse, c'est-à-dire que le lien doit être disponible à chaque t_i tel que $t_i \in [t, t+p]$.



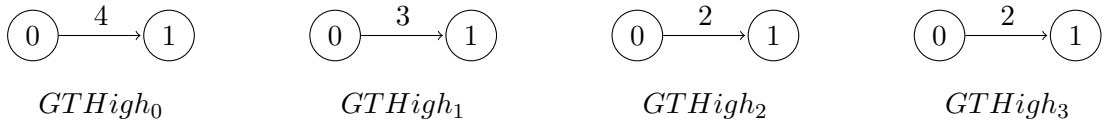
GT_0 GT_1 GT_2 GT_3

Lien de base



$GTLow_0$ $GTLow_1$ $GTLow_2$ $GTLow_3$

Impact Bas



$GTHigh_0$ $GTHigh_1$ $GTHigh_2$ $GTHigh_3$

Impact Haut

4.2.2 Algorithme de calcul de la mesure

Comme dit précédemment, nous avons adapté l'algorithme de Dijkstra pour les graphes temporels. Avec la nouvelle structure de données, la seule différence avec le dijkstra classique est qu'au lieu de regarder la distance avec le noeud actuel, on regarde la distance avec le noeud voisin au temps déjà parcouru.

TemporalDijkstra(graphe temporel GT, Noeud depart, Noeud arrivee) -> Distance

```
NoeudVisited ← ∅
NoeudsATraiter ← {depart}
while nodesToVisit ≠ {} do
    Noeud currentNode ← nodesToVisit.pop()
    if currentNode is arrivee then
        return visited[currentNode]
    end if
    visited[currentNode] ← distance
    PhantomEdges neighbours ← GT.getNeighbours(currentNode)
    for voisin in neighbours do
        distance dist ← visited[currentNode] + voisin[visited[currentNode]]
        if voisin not in visited or visited[voisin] > dist then
            visited[voisin] ← dist
            nodesToVisit.push(voisin)
        end if
    end for
end while
```

---

[4]

A noter que cet algorithme reste correct pour tout type de graphe temporel (discret) ou flot de liens (continu), multi-graphe[5] ou non, orienté ou non, tant qu'il vérifie ces 2 propriétés : Pour tout noeud u, v, ayant pour poids au cours du temps $p_{u,v}(t)$,

- $p_{u,v}(t) > 0 \ \forall t \in \mathbb{R}$, car l'algorithme de Dijkstra ne peut pas travailler avec des poids négatifs.
- $\frac{p_{u,v}(t)}{p_{u,v}(t')} \geq -1 \ \forall t, t' \in \mathbb{R} | t > t'$, car notre algorithme suppose qu'une attente n'est jamais bénéfique.

Mettre au propre temporal dijkstra

4.2.3 Optimisations faites

L'algorithme de Dijkstra est donc mis en place et adapté pour les graphes temporels, mais il reste encore des optimisations à faire. En effet, l'algorithme de Dijkstra est souvent utilisé pour calculer le plus court chemin entre deux noeuds, mais il calcule en réalité beaucoup plus de plus courts chemins. Donc nous avons modifié l'algorithme pour qu'il écrive directement dans la matrice de distances, puis le but va être d'écrire le plus de distances possibles en une seule fois.

Nous avons prit parti d'optimiser pour le temps et non la mémoire, le projet étant fait pour tourner sur des supercalculateurs avec des centaines de Go de RAM. Ainsi que le temps global de plusieurs

4. L'algorithme est simplifié et renvoie une distance et non un chemin, mais il est facile de le modifier pour qu'il renvoie un chemin.

5. Graphe ayant plusieurs liens entre 2 noeuds

attaques plutôt que le temps d'une seule attaque, le but du projet étant de regarder une évolution de l'efficacité des attaques.

Calcul de la distance entre le noeud de départ et tous les autres noeuds

Pour connaître la distance entre le noeud de départ et le noeud d'arrivée, Dijkstra va explorer beaucoup de noeuds candidats. Donc, pour chaque noeud candidat à une distance d du noeud de départ qu'il sort du tas, on va noter cette distance dans la matrice de distances.

Prise en compte de tous les sous-chemins

Soit un chemin calculé 0->1->2->3, on a un chemin trivial à noter entre 0 et 3, mais on a aussi beaucoup de sous-chemins qui sont calculés. Ici, on a les chemins 0->1->2, 1->2->3, 0->1, 1->2, 2->3.

Mettre au propre le chemin

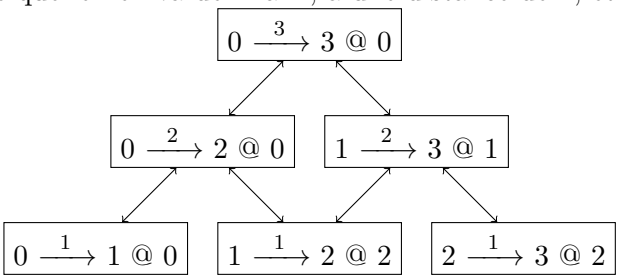
Valeur à l'index 2 : two

Et en général, pour un chemin de taille n, on a $n(n-1)/2$ sous-chemins qu'on peut noter dans la matrice de distances.

Nous avons donc créer une structure de données qui va permettre de rapidement stocker tous les sous-chemins à partir des pas faits dans le chemin. Cette structure ressemble à un arbre de segments, mais en structure pyramidale.

Les éléments $0 \xrightarrow{1} 1$, $1 \xrightarrow{1} 2$, $2 \xrightarrow{1} 3$ sont les briques de base de la pyramide, qui sont l'équivalent des feuilles dans un arbre. Dans notre pyramide, chaque élément va stocker le noeud de départ, le noeud d'arrivée, la distance entre ces deux noeuds, et le temps nécessaire pour arriver au noeud de départ. Ensuite, pour chaque étage de la pyramide, on va regarder notre fils gauche et notre fils droit, ainsi que notre petit-fils commun. Le noeud de départ sera celui du fils gauche, le noeud d'arrivée celui du fils droit, la distance sera la somme des distances des deux fils moins la distance du petit-fils (partie commune comptée deux fois sinon), et le temps nécessaire pour arriver au noeud de départ sera celui du fils gauche. S'il n'y a pas de petit-fils, alors il n'y a pas de distance commune, et donc la distance sera la somme des distances des deux fils.

Pour l'exemple précédent, notre pyramide ressemblerait à ça : Avec la notation A $\xrightarrow{x}$ B @ t pour dire que le lien va de A à B, a une distance de x, et qu'il faut t ticks pour arriver à A.



Prédiction des chemins avec le plus d'étapes

Cette optimisation va de pair avec la précédente, en faisant des benchmarks, nous avons remarqué que parcourir la boucle j à l'envers permettait de gagner du temps. On suppose que c'est dû au fait que les noeuds avec des indices éloignés soient aussi éloignés dans le graphe, donc que le chemin entre ces deux noeuds ai plus d'étapes, et que donc on va écrire plus de sous-chemins dans la matrice de distances qui seraint écrasés plus tard si on avait parcouru la boucle dans le sens normal.

Regarder ses voisins

La distance de u à v est le minimum entre la distance de u à k plus la distance de k à v pour chaque noeud k tel que l'arête $u->k$ et l'arête $k->v$ existent. Donc si nous connaissons toutes ces distances $u->k$, alors nous pouvons calculer la distance $u->v$ sans avoir besoin de reparcourir le graphe.

Nous avons utilisé cette propriété pour optimiser notre algorithme. Tout d'abord, nous créons le graphe inverse, qui pour chaque arête $u->v$ du graphe original va créer une arête $v->u$.

Ensuite, pour avoir la distance entre u et v, nous cherchons tous les noeuds k tels que l'arête $k->v$ existe grâce au graphe inverse. Pour chaque noeud k, si la distance est calculée, alors nous pouvons directement la récupérer, sinon nous la calculons. Ensuite, nous prenons le minimum de toutes ces distances ($u->k+k->v$) pour avoir la distance $u->v$. C'est très utile pour les noeuds qu'on ne peut atteindre que par un seul autre noeud, car ça évite de refaire un parcours avec juste un noeud en plus.

Se rapporter à un parcours de graphe classique quand c'est possible

Certaines stratégies d'attaque sont statiques, et ne dépendent pas du temps. Cela implique que toutes les matrices de distances sont les mêmes, et donc que les calculer pour chaque matrice est inutile. Ainsi, on a divisé le format d'attaque en deux, un format statique et un format dynamique, et lorsque l'algorithme détecte que l'attaque est statique, il va se rapporter à un parcours de graphe classique avec une seule matrice de distances, puis calculer l'efficacité de l'attaque comme s'il y en avait plusieurs.

Ref vers attaques

Calcul du non-attqué

Nous nous sommes rendus compte que les matrices de distances plus loin dans le temps étaient beaucoup plus vides que les premières, dû à la nature de la mesure, qui va ignoré tous les chemins impossible à compléter dans notre lapse de temps. Par exemple, si notre tick max est de 100, notre tick courant est de 70, alors on ne peut pas calculer un chemin de plus de 30 ticks. Surtout que ne pas pouvoir le calculer signifie qu'on ne peut pas écrire plusieures valeurs dans la matrice de distances car il n'y a pas de chemin complet. Donc, on commence par calculer les distances sur le graphe non-attaqué, grâce à la méthode précédante, puis on dit au programme de ne calculer des chemins qui ont une chance de finir dans le temps imparti.

4.2.4 Complexité de l'algorithme

Donc même si la compléxité pire cas de faire des Dijkstra pour chaque noeud est en $O(n^3 \log n * t)$, la complexité amortie est bien meilleure. La calculer réellement serait extrêmement complexe et dépend beaucoup de la topologie du graphe, mais sur des villes réelles, nous avons mesurer l'évolution du temps de calcul en fonction du nombre de noeuds, et nous avons obtenu cette courbe : qui peut être modélisée par une fonction en $O(???????)$.

Courbe de la complexité

4.3 Autres optimisations tentées : Invalidation des chemins attaqués

Calcul de la mesure

Changer le nom

Afin de réduire le nombre de chemins à calculer, nous avons tenté de calculer une première fois les plus courts chemins, puis à chercher à invalider les chemins comportant un lien attaqué, aux temps où celui-ci est attaqué et réutiliser les résultats non impactés. Nous avons déterminé que si un lien (u, v) est attaqué au temps t, alors

- Tout chemin commençant par le lien attaqué est invalidé au temps t
- Les chemins de la forme $(x, ..., u, v, ..., y)$n avec $(x, ..., u)$ un sous chemins de distance w, sont invalidés au temps $t - w$

Première invalidation

label

Nous avons dans un premier temps invalidé les chemins en utilisant les pyramides définies dans . Pour cela nous associons à chaque sous-chemin des ticks pendant lesquels ils seront invalidés, vide au début, sauf pour le dernier niveau (correspondant aux liens) où cela corresponds aux ticks où le lien est attaqué. Puis, en partant du bas, on propage aux deux sous-chemins au-dessus.

expliquer la méthode

mettre algo ?

Algorithm 7 Première invalidation

```
function init_intervals_to_invalidate(Path path, Attack attack)
    intervals_to_invalidate ← Dictionnary<SubPath,Set<Tick>>
    for all subpath ∈ path.leaves] do
        if subpaths.is_unvalidated then
            for time ∈ time_matrix[subpath.start, subpath.end] do
                for delta ∈ ⟦0, attack.delta − 1⟧ do
                    idx ← time × attack.delta + delta
                    if idx < attack.max_time then
                        interval_to_invalidate[subpath].insert(idx)
                    end if
                end for
            end for
        end if
    end for
    return intervals_to_invalidate
end function
intervals_to_invalidate ← init_intervals_to_invalidate(path, attack)
for all subpath ∈ path.subpaths.rev() do
    if subpath.has_left_parent then
        left ← subpath.left_parent
        for time ∈ intervals_to_invalidate[subpath] do
            intervals_to_invalidate[left].insert(time − (subpath.time_taken − left.time_taken))
        end for
    end if
    if subpath.has_right_parent then
        right ← subpath.right_parent
        for time ∈ intervals_to_invalidate[subpath] do
            intervals_to_invalidate[right].insert(time)
        end for
    end if
    for time ∈ interval_to_invalidate[subpath] do
        dma[time][subpath.start][subpath.end] ← 0
    end for
end for
```

Deuxième invalidation

Cette première méthode c'est avérée peu efficace. En effet, dans le cas où deux chemins partagent un même sous-chemin, le calcul et la propagation des temps où le sous-chemin sera invalidé sera fait deux fois. Nous avons donc choisis d'utiliser un tableau en 2 dimensions indexé par un couple (u, v), correspondant au chemin allant de u à v. Chaque case de ce tableau contient la distance de u à v, les sommets sources et de destinations, ainsi que les sommets x tels que $(x, u, ..., v)$ soit le plus court chemin de x à v et y tels que $(u, ..., v, y)$ soit le plus court chemin de u à y.

L'invalidation reprends l'algorithme de l'invalidation précédente, à la différence que l'on utilise une file contenant initialement que les liens du graphe au lieu d'itérer sur un tableau, et que l'on ne traite un chemin qu'une seule fois, puis on ajoute les chemins à traiter dans la file.

mettre algo ?

Algorithm 8 Seconde invalidation

```
intervals_to_invalidate ← init_intervals_to_invalidate(path, attack)
while !to_invalidate.is_empty do
    (start, end) ← fifo.pop()
    subpath ← path_matrix[start, end].subpath
    for all left_node ∈ path_matrix[start, end].left_neighbours do
        left ← path_matrix[left_node, end].subpath
        for time ∈ intervals_to_invalidate[subpath] do
            intervals_to_invalidate[left].insert(time − (subpath.time_taken − left.time_taken))
        end for
    end for
    for all right_node ∈ path_matrix[start, end].right_neighbours do
        right ← path_matrix[right_node, end].subpath
        for time ∈ intervals_to_invalidate[subpath] do
            intervals_to_invalidate[right].insert(time)
        end for
    end for
    for time ∈ interval_to_invalidate[subpath] do
        dma[time][subpath.start][subpath.end] ← 0
    end for
end while
```

expliquer pourquoi on ne l'a pas gardé

5 Résultats et interprétation

Résultats et interprétation

La figure de gauche, montrant l'évolution de l'efficacité en fonction du nombre de liens retirés pour différentes stratégie d'attaque montre que la modélisation "gentille" de l'attaque est peu efficace. En effet, les stratégies profitant du dynamisme du graphe sont bien moins efficace que des stratégies d'attaque statiques. Le fait que les stratégies d'attaque dynamiques soient moins efficaces dans la modélisation "gentille" de l'attaque est probablement dû au fait que peu de chemins sont vraiment bloqués par les attaques, en effet, les chemins supprimés au temps T sont souvent rétablis au temps T+1. Cela fait que l'impact des attaques dynamiques est faible par rapport aux attaques statiques, qui bloquent des chemins de manière permanente.

La figure de droite, montrant l'évolution de l'efficacité en fonction du nombre de liens retirés pour différentes stratégie d'attaque montre que la modélisation "agressive", bien que toujours moins efficace que les attaques statiques, est bien plus efficace que la modélisation "gentille". Chaque lien supprimé lors d'une attaque agressive bloquera les chemins dont il fait partie pendant un temps égal au poids du lien. Les attaques agressives ont un impact plus fort sur l'efficacité.

courbe pas agressive et agressive me-lun ici

On en conclut donc qu'avec un changement mineur dans la modélisation des attaques, on a des résultats très différents. Avec notre modélisation actuelle de l'efficacité sur un graphe dynamique et notre modélisation des attaques, les attaques dynamiques restent moins impactantes que les attaques statiques. Cependant, la différence d'impact entre la modélisation "gentille" et "agressive" est très importante.

Cela montre qu'il est utile de continuer à travailler sur une mesure de l'efficacité conçue pour les graphes dynamiques et possiblement sur une modélisation plus fine des attaques sur un réseau urbain.

définir attaque dynamique / statique si pas fait plus haut

Ces figures montrent l'évolution de l'impact des attaques dynamiques sur l'efficacité en fonction du nombre de "ticks" avant le changement de liens attaqués. Le but de ces figures était de déterminer l'impact du nombre de "ticks" sur l'efficacité des attaques. Ces figures nous montre qu'un nombre de tick trop bas à un impact plus faible. Cela rejoint le problème des stratégies "gentilles" qui ont un impact faible sur l'efficacité car les chemins supprimés sont souvent rétablis rapidement.

6 Perspectives et ouvertures

changer nom stratégie gentille / agres-

dire -> on pourrait tester d'autres stratégies d'attaque -> par exemple sur graphe "segmenté/ discretisé" -> produire des données sur plusieurs gdes villes -> étudier des mesures adjacentes (reachability etc) -> tweak le modèle d'attaque

7 Conclusion

Ce projet a permis de définir et tester une mesure de l'efficacité adaptée à un graphe dynamique. Il a également permis de définir des modélisations de manifestations comme des attaques sur un graphe dynamique.

Les résultats obtenus sont prometteurs et mettent en évidence l'influcne de l'aspect dynamique du graphe sur l'efficacité des stratégies d'attaque.

On pourrait cependant changer la modélisation de notre problème, ajuster la mesure de l'efficacité ou encore tester d'autres stratégies d'attaque pour obtenir de meilleurs résultats.

Cependant, ce projet a permis de mettre en lumière le fait qu'il était nécessaire de créer et tester des mesures adaptées aux graphes dynamiques afin de pouvoir utiliser ceux-ci pour modéliser des problèmes réels.

Terminol

Conclusi

en vrai je mens un peu la ça marche pas si bien, je sais pas quoi dire

peut etre mauvais, pk on l'a pas fait dans ce cas ?

je pense c bien dans l'idée mais peut etre mal formulé

Références